

Sort a Linked List of 0s, 1s and 2s

Structured DSA Study Notebook

This notebook follows the uploaded TUF transcript. The transcript first explains data replacement as the brute-force/two-pass approach and then optimizes it by changing links in one traversal. ■filecite■turn3file0■L75-L105■

1. Introduction

The linked list contains only 0, 1 and 2. The task is to sort the list by data so that all 0s come first, followed by 1s, followed by 2s, and return the updated head. ■filecite■turn3file0■L19-L65■

2. Problem Statement

Input:
head -> [0/1/2] -> [0/1/2] -> ... -> NULL

Requirement:
0s -> 1s -> 2s -> NULL

Return:
Updated head of the sorted linked list

3. Required Concepts

3.1 Counting

count0 = number of nodes containing 0
count1 = number of nodes containing 1
count2 = number of nodes containing 2

The transcript uses one traversal to count the three values and a second traversal to replace the node data. ■filecite■turn3file0■L367-L467■

3.2 Dummy Nodes

dummy0 -> list containing 0s
dummy1 -> list containing 1s
dummy2 -> list containing 2s

zero -> last node of 0-list
one -> last node of 1-list
two -> last node of 2-list

The optimized method uses three dummy nodes and three moving pointers to build three segregated lists during one traversal. ■filecite■turn3file0■L795-L879■

4. Approaches

Approach 1: count 0s, 1s and 2s, then replace data. Approach 2: preserve the existing nodes and rearrange their links into three lists, then connect the lists. ■filecite■turn3file0■L587-L655■

5. Approach 1 — Data Replacement

5.1 Idea

1. Traverse the list.
2. Count 0s, 1s and 2s.
3. Traverse the list again.
4. Replace data with all 0s first.
5. Then replace data with all 1s.
6. Then replace data with all 2s.
7. Return head.

The transcript explicitly describes this as the initial brute-force/data-replacement approach. ■filecite■turn3file0■L75-L199■

5.2 Pseudocode

```
START

count0 = 0
count1 = 0
count2 = 0

temp = head

WHILE temp != NULL

    IF temp.data == 0
        count0++
    ELSE IF temp.data == 1
        count1++
    ELSE
        count2++

    temp = temp.next

END WHILE

temp = head

WHILE temp != NULL

    IF count0 > 0
        temp.data = 0
        count0--
    ELSE IF count1 > 0
        temp.data = 1
        count1--
    ELSE
        temp.data = 2
        count2--

    temp = temp.next

END WHILE

RETURN head

END
```

5.3 Original Core Code From Transcript

```
// First traversal
temp = head;

while(temp != null){

    if(temp.data == 0){
        count0++;
    }
    else if(temp.data == 1){
        count1++;
    }
    else{
        count2++;
    }

    temp = temp.next;
}

// Second traversal
temp = head;

while(temp != null){

    if(count0 > 0){
        temp.data = 0;
        count0--;
    }

}
```

```

else if(count1 > 0){
    temp.data = 1;
    count1--;
}
else{
    temp.data = 2;
    count2--;
}

temp = temp.next;
}

```

These operations correspond to the transcript's counting traversal and data-replacement traversal.

■filecite■turn3file0■L395-L467■ ■filecite■turn3file0■L475-L585■

5.4 Complete Java Code

```

class Solution {

    public ListNode sortList(ListNode head) {

        // Handle an empty list
        if(head == null){
            return null;
        }

        // Count the number of 0s
        int count0 = 0;

        // Count the number of 1s
        int count1 = 0;

        // Count the number of 2s
        int count2 = 0;

        // Start the first traversal
        ListNode temp = head;

        // Traverse the complete list
        while(temp != null){

            // Count a zero
            if(temp.data == 0){
                count0++;
            }

            // Count a one
            else if(temp.data == 1){
                count1++;
            }

            // Count a two
            else{
                count2++;
            }

            // Move to the next node
            temp = temp.next;
        }

        // Start the second traversal
        temp = head;

        // Replace node data
        while(temp != null){

            // Place a zero if zeros remain
            if(count0 > 0){
                temp.data = 0;
                count0--;
            }

            // Otherwise place a one if ones remain

```

```

        else if(count1 > 0){
            temp.data = 1;
            count1--;
        }

        // Otherwise place a two
        else{
            temp.data = 2;
            count2--;
        }

        // Move to the next node
        temp = temp.next;
    }

    // Return the original head
    return head;
}

```

5.5 Static Visual Representation

Original:

```

HEAD
|
v
[1] -> [0] -> [2] -> [0] -> [1] -> [2] -> NULL

```

Count:

```

0s = 2
1s = 2
2s = 2

```

Data replacement:

```

HEAD
|
v
[0] -> [0] -> [1] -> [1] -> [2] -> [2] -> NULL

```

Node links remain the same.
Only node data is changed.

5.6 Complexity

Time:
 $O(n)$

Space:
 $O(1)$

Reason:
Two complete traversals are performed.
 $O(2n)$ becomes $O(n)$.

The transcript describes the two traversals as $O(2N)$, which is represented in Big-O notation as $O(N)$, and uses $O(1)$ extra space. ■filecite■turn3file0■L587-L619■

6. Approach 2 — One-Pass Link Rearrangement

6.1 Why Optimize?

The transcript points out that the data-replacement solution uses two passes and then asks for a one-pass solution. The alternative is to change the given links instead of changing node data. ■filecite■turn3file0■L619-L655■

6.2 Idea

Create three segregated lists:

```
0-list
dummy0 -> 0 -> 0 -> 0 -> NULL
```

```
1-list
dummy1 -> 1 -> 1 -> NULL
```

```
2-list
dummy2 -> 2 -> 2 -> NULL
```

Traverse the original list once.

```
For each node:
    if data == 0 -> attach to 0-list
    if data == 1 -> attach to 1-list
    if data == 2 -> attach to 2-list
```

```
Finally:
0-list -> 1-list -> 2-list
```

The transcript describes this as storing the zeros, ones and twos in separate linked lists and connecting them at the end. ■filecite■turn3file0■L695-L759■

6.3 Pseudocode

```
START

IF head == NULL OR head.next == NULL
    RETURN head

Create dummy0
Create dummy1
Create dummy2

Set zero = dummy0
Set one = dummy1
Set two = dummy2

temp = head

WHILE temp != NULL

    IF temp.data == 0

        zero.next = temp
        zero = zero.next

    ELSE IF temp.data == 1

        one.next = temp
        one = one.next

    ELSE

        two.next = temp
        two = two.next

    temp = temp.next

END WHILE

IF dummy1.next != NULL
```

```

    zero.next = dummy1.next
ELSE
    zero.next = dummy2.next

one.next = dummy2.next
two.next = NULL

RETURN dummy0.next

END

```

6.4 Original Core Code From Transcript

```

// Attach a zero node
zero.next = temp;

// Move zero pointer
zero = zero.next;

// Attach a one node
one.next = temp;

// Move one pointer
one = one.next;

// Attach a two node
two.next = temp;

// Move two pointer
two = two.next;

// Move traversal pointer
temp = temp.next;

// Connect zero list to one list when present
zero.next = oneHead;

// Otherwise connect zero list to two list
zero.next = twoHead;

// Connect one list to two list
one.next = twoHead;

// Get the new head
return zeroHead.next;

```

The transcript explains the pointer movement and the final connections between the zero, one and two lists, including the cases where one of the lists is empty. ■filecite■turn3file0■L1421-L1545■ ■filecite■turn3file0■L1561-L1715■

6.5 Complete Java Code — Line by Line

```

class Solution {

    public ListNode sortList(ListNode head) {

        // Handle an empty list
        if(head == null){
            return head;
        }

        // Handle a single-node list
        if(head.next == null){
            return head;
        }

        // Create dummy node for the 0-list
        ListNode zeroHead = new ListNode(-1);

        // Create dummy node for the 1-list
        ListNode oneHead = new ListNode(-1);

        // Create dummy node for the 2-list
        ListNode twoHead = new ListNode(-1);
    }
}

```

```

// Pointer at the end of the 0-list
ListNode zero = zeroHead;

// Pointer at the end of the 1-list
ListNode one = oneHead;

// Pointer at the end of the 2-list
ListNode two = twoHead;

// Start traversal from the original head
ListNode temp = head;

// Traverse the original list once
while(temp != null){

    // Save the next original node
    ListNode next = temp.next;

    // Separate the current node from the old link
    temp.next = null;

    // Attach current node to the 0-list
    if(temp.data == 0){

        // Connect current node
        zero.next = temp;

        // Move zero pointer
        zero = zero.next;
    }

    // Attach current node to the 1-list
    else if(temp.data == 1){

        // Connect current node
        one.next = temp;

        // Move one pointer
        one = one.next;
    }

    // Attach current node to the 2-list
    else{

        // Connect current node
        two.next = temp;

        // Move two pointer
        two = two.next;
    }

    // Continue with the original next node
    temp = next;
}

// Connect the 0-list to the 1-list if it exists
if(oneHead.next != null){
    zero.next = oneHead.next;
}

// Otherwise connect the 0-list to the 2-list
else{
    zero.next = twoHead.next;
}

// Connect the 1-list to the 2-list
one.next = twoHead.next;

// End the 2-list explicitly
two.next = null;

// Return the first real node after dummy0
return zeroHead.next;

```

```
    }
}
```

6.6 Static Visual Representation

Original:

```
HEAD
|
v
[1] -> [0] -> [2] -> [0] -> [1] -> [2] -> NULL
```

After one traversal:

```
ZERO LIST
dummy0 -> [0] -> [0] -> NULL
           ^
         zero
```

```
ONE LIST
dummy1 -> [1] -> [1] -> NULL
           ^
         one
```

```
TWO LIST
dummy2 -> [2] -> [2] -> NULL
           ^
         two
```

Final connection:

```
ZERO LIST -> ONE LIST -> TWO LIST

HEAD
|
v
[0] -> [0] -> [1] -> [1] -> [2] -> [2] -> NULL
```

6.7 Why Save temp.next?

```
// Current node
temp

// Save its original next node
ListNode next = temp.next;

// Disconnect current node from old list
temp.next = null;

// Attach current node to its category list

// Continue original traversal
temp = next;
```

This keeps the traversal pointer safe while the current node's link is changed.

6.8 Linking the Three Lists

```
// If 1-list exists:
zero.next = oneHead.next;

// If 1-list does not exist:
zero.next = twoHead.next;

// Connect 1-list to 2-list:
one.next = twoHead.next;

// End 2-list:
two.next = null;
```

The transcript explicitly discusses the cases where the 1-list or 2-list is absent and explains why the final links must still produce a valid sorted list. ■filecite■turn3file0■L1561-L1715■

7. Edge Cases

Case 1: Empty list
head = NULL
return head

Case 2: One node
[1] -> NULL
return head

Case 3: Only zeros
[0] -> [0] -> [0]
0-list contains all nodes

Case 4: Only ones
[1] -> [1] -> [1]
1-list contains all nodes

Case 5: Only twos
[2] -> [2] -> [2]
2-list contains all nodes

Case 6: No zeros
0-list is empty.
Use the 1-list or 2-list as the new starting list.

Case 7: No ones
Connect the 0-list directly to the 2-list.

Case 8: No twos
The 2-list remains empty and ends at NULL.

The transcript explicitly asks the learner to test examples containing only zeros, zeros and ones, and ones and twos, and explains the empty-list connections. ■filecite■turn3file0■L1735-L1771■ ■filecite■turn3file0■L2003-L2027■

8. Complexity Comparison

Approach	Traversal	Time	Extra Space	Changes
Data Replacement	2 passes	$O(n)$	$O(1)$	Node data
Link Rearrangement	1 pass	$O(n)$	$O(1)$	Node links

The transcript describes the optimized method as a one-pass solution using the same nodes, with $O(n)$ time and $O(1)$ auxiliary space. ■filecite■turn3file0■L2051-L2085■

9. Quick Revision

Approach 1:
1. Count 0, 1 and 2.
2. Traverse again.
3. Replace data.
4. $O(n)$ time.
5. $O(1)$ extra space.

Approach 2:
1. Create dummy0, dummy1 and dummy2.
2. Keep zero, one and two pointers.
3. Traverse once.
4. Attach each node to its value list.
5. Connect 0-list to 1-list or 2-list.
6. Connect 1-list to 2-list.
7. End 2-list with NULL.
8. Return zeroHead.next.
9. $O(n)$ time.
10. $O(1)$ extra space.

10. Conclusion

The transcript progresses from a two-pass data-replacement method to a one-pass link-rearrangement method. The important interview idea is to recognize that the values are restricted to 0, 1 and 2, then understand how three temporary list heads and moving pointers can segregate the existing nodes before joining the three lists.

■filecite■turn3file0■L619-L759■